Week 42: From Backpropagation to Differential Equations

Morten Hjorth-Jensen
Viktor Svensson

Department of Physics and Center for Computing in Science Education,
University of Oslo, Norway

Plan for today

First hour

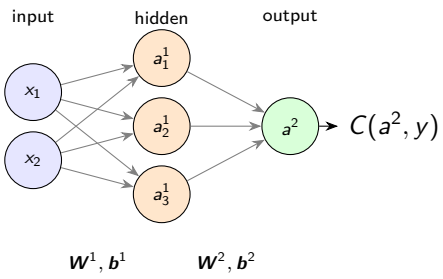
1. **Backpropagation**: the chain rule, organised
2. **Activation and cost functions**: why they come in pairs
3. **Universal approximation**: what a network can represent

Second hour

4. **Neural networks for differential equations**
   - the equation as the cost
   - why the activation function matters
   - boundary conditions
   - pros and cons vs. standard methods

Tuesday: backpropagation in code. **Exercises week 42:** deadline Friday October 16.

Recap: the forward pass



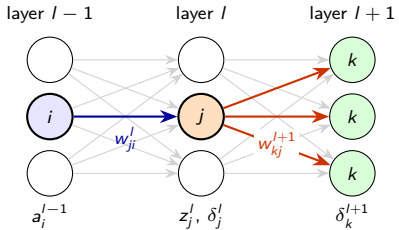
One sample $\boldsymbol{x}$, layers $l = 1, \ldots, L$, $\boldsymbol{a}^0 = \boldsymbol{x}$:

$$\boldsymbol{z}^l = \boldsymbol{W}^l \boldsymbol{a}^{l-1} + \boldsymbol{b}^l, \qquad \boldsymbol{a}^l = f(\boldsymbol{z}^l)$$

- $\boldsymbol{W}^l$ has shape (nodes out, nodes in)
- f acts element by element; the last layer has its own *output* activation f_L
- the cost C compares the output $\boldsymbol{a}^L$ with the target $\boldsymbol{y}$

In code with many samples we stack them as rows, $\boldsymbol{Z}^l = \boldsymbol{A}^{l-1}\boldsymbol{W}^l + \boldsymbol{b}^l$, so $\boldsymbol{W}^l$ is stored as (in, out): the same equations, transposed.

Backpropagation: the chain rule, organised



Node j in layer l, one sample:

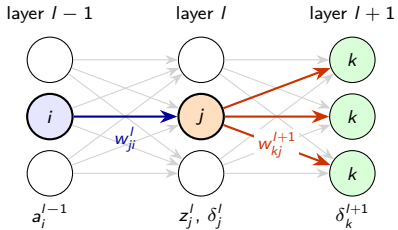
$$z_j^l = \sum_i w_{ji}^l\, a_i^{l-1} + b_j^l, \qquad a_j^l = f(z_j^l)$$

The *error* of node j is $\delta_j^l = \dfrac{\partial C}{\partial z_j^l}$.

On the board: Chain rule

Chain rule: δ_j^L, then δ_j^l from all the δ_k^{l+1}, then $\partial C/\partial w_{ji}^l$ and $\partial C/\partial b_j^l$.

Backpropagation: the chain rule, organised



Node j in layer l, one sample:

$$z_j^l = \sum_i w_{ji}^l\, a_i^{l-1} + b_j^l, \qquad a_j^l = f(z_j^l)$$

The *error* of node j is $\delta_j^l = \dfrac{\partial C}{\partial z_j^l}$.

On the board: Chain rule

Chain rule: δ_j^L, then δ_j^l from all the δ_k^{l+1}, then $\partial C/\partial w_{ji}^l$ and $\partial C/\partial b_j^l$.

Result

$$\delta_j^L = f_L'(z_j^L)\,\frac{\partial C}{\partial a_j^L}, \qquad \delta_j^l = \Big(\sum_k w_{kj}^{l+1}\,\delta_k^{l+1}\Big) f'(z_j^l), \qquad \frac{\partial C}{\partial w_{ji}^l} = \delta_j^l\, a_i^{l-1}, \qquad \frac{\partial C}{\partial b_j^l} = \delta_j^l$$

- Start at the output, go *backwards*: each δ^l reuses δ^{l+1}.
- Node j feeds *every* node k of layer $l+1$, so its error comes back along every edge: the sum over k.
- Weight gradient = error at the end of the edge $\times$ signal at its start. The error picks up a factor $w\,f'$ in every layer.

The general case: a batch of samples

As in code: stack the n samples as **rows**, $\boldsymbol{A}^0 = \boldsymbol{X}$ of shape (n, n_0), and store $\boldsymbol{W}^l$ as (in, out), shape (n_{l-1}, n_l):

$$\boldsymbol{Z}^l = \boldsymbol{A}^{l-1}\boldsymbol{W}^l + \boldsymbol{b}^l, \qquad \boldsymbol{A}^l = f(\boldsymbol{Z}^l),$$

with $\boldsymbol{b}^l$ added to every row. The cost is the mean over the batch, $C = \frac{1}{n}\sum_s C_s$, and the errors form a matrix with one row per sample s:

$$\Delta^l_{sj} = \frac{\partial C}{\partial Z^l_{sj}}.$$

The $\frac{1}{n}$ of the mean sits in $\partial C/\partial \boldsymbol{A}^L$; $\circ$ is the element-wise product.

Backpropagation for a batch

$$\boldsymbol{\Delta}^L = f'_L(\boldsymbol{Z}^L) \circ \frac{\partial C}{\partial \boldsymbol{A}^L}$$

$$\boldsymbol{\Delta}^l = \left(\boldsymbol{\Delta}^{l+1}(\boldsymbol{W}^{l+1})^T\right) \circ f'(\boldsymbol{Z}^l)$$

$$\frac{\partial C}{\partial \boldsymbol{W}^l} = (\boldsymbol{A}^{l-1})^T\boldsymbol{\Delta}^l$$

$$\frac{\partial C}{\partial \boldsymbol{b}^l} = \sum_s \boldsymbol{\Delta}^l_{s,:} \quad \text{(sum of the rows)}$$

- The one-sample equations, transposed: samples are rows.
- Shapes: $\boldsymbol{\Delta}^l$ is (n, n_l), and $(\boldsymbol{A}^{l-1})^T\boldsymbol{\Delta}^l$ is (n_{l-1}, n_l), the shape of $\boldsymbol{W}^l$.
- The matrix product sums over the samples: the gradient of the whole batch in one operation per layer.

Quick question 1: why not start at zero?

Question

A network starts with *all* weights and biases equal to zero. Look at the equations: which gradients are non-zero after the first backward pass? What happens after that?

$$\boldsymbol{\delta}^L = f_L'(\boldsymbol{z}^L) \circ \frac{\partial C}{\partial \boldsymbol{a}^L}$$

$$\boldsymbol{\delta}^l = \left((\boldsymbol{W}^{l+1})^T \boldsymbol{\delta}^{l+1}\right) \circ f'(\boldsymbol{z}^l)$$

$$\frac{\partial C}{\partial \boldsymbol{b}^l} = \boldsymbol{\delta}^l, \qquad \frac{\partial C}{\partial \boldsymbol{W}^l} = \boldsymbol{\delta}^l \, (\boldsymbol{a}^{l-1})^T$$

Quick question 1: why not start at zero?

Question

A network starts with *all* weights and biases equal to zero. Look at the equations: which gradients are non-zero after the first backward pass? What happens after that?

$$\boldsymbol{\delta}^L = f_L'(\boldsymbol{z}^L) \circ \frac{\partial C}{\partial \boldsymbol{a}^L}$$

$$\boldsymbol{\delta}^l = \left((\boldsymbol{W}^{l+1})^T \boldsymbol{\delta}^{l+1}\right) \circ f'(\boldsymbol{z}^l)$$

$$\frac{\partial C}{\partial \boldsymbol{b}^l} = \boldsymbol{\delta}^l, \qquad \frac{\partial C}{\partial \boldsymbol{W}^l} = \boldsymbol{\delta}^l \, (\boldsymbol{a}^{l-1})^T$$

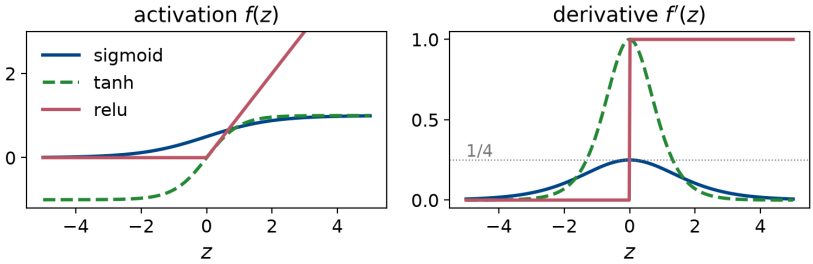
Answer

Only the **output layer**.

- $\boldsymbol{W}^{l+1} = \boldsymbol{0}$ gives $\boldsymbol{\delta}^l = \boldsymbol{0}$ in every hidden layer, whatever f is: no hidden weight or bias moves.
- Learning has to creep backwards from the output, one layer per step, and every gradient is a product of near-zero weights. Slow to start at best; if $f(0) = 0$ the network never leaves $\boldsymbol{W} = \boldsymbol{0}$.
- **Symmetry**: all units in a layer start identical, so they get identical gradients and stay copies of each other. The same happens for any equal non-zero start.

Random weights break the symmetry.

Hidden activation functions



Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$
$\sigma' = \sigma(1-\sigma) \leq \frac{1}{4}$.
Flat ($\sigma' \approx 0$) for large $|z|$.

tanh
$\tanh' = 1 - \tanh^2 \leq 1$.
Centred at zero; also flat for large $|z|$.

ReLU $\max(0, z)$
$f' = 1$ for $z > 0$, 0 otherwise. A unit with $z < 0$ for all inputs never learns ("dead").

What matters

f must be **non-linear**: otherwise the whole network is a single linear map. For training, f' matters. Backpropagation multiplies by f' once per layer, so with L sigmoid layers the f' factors alone multiply the error by at most $(\frac{1}{4})^L$: the *vanishing gradient*. This is why ReLU is the default in deep networks.

Initialising the weights

Random, to break the symmetry. **The scale matters.** $\boldsymbol{z} = \boldsymbol{W}\boldsymbol{a} + \boldsymbol{b}$ with M inputs, and independent zero-mean entries of variance σ_w^2 in $\boldsymbol{W}$ and σ_b^2 in $\boldsymbol{b}$:

$$\mathbb{E}[\boldsymbol{z}\boldsymbol{z}^T] = \mathbb{E}[\boldsymbol{W}\boldsymbol{a}\boldsymbol{a}^T\boldsymbol{W}^T] + \mathbb{E}[\boldsymbol{b}\boldsymbol{b}^T] = \left(M\sigma_w^2\,\overline{a^2} + \sigma_b^2\right)\boldsymbol{I}$$

With $\boldsymbol{b} = \boldsymbol{0}$, $\mathrm{Var}(z) = M\sigma_w^2\,\overline{a^2}$, where $\overline{a^2}$ is the mean squared *incoming* activation. For tanh near zero $a \approx z$, so each layer multiplies the variance by $M\sigma_w^2$.

- σ_w too large: $|z|$ large, tanh/sigmoid saturate, gradients blow up or die.
- σ_w too small: the signal shrinks in every layer, and so do the gradients.

Rule: keep $\mathrm{Var}(z)$ the same in every layer

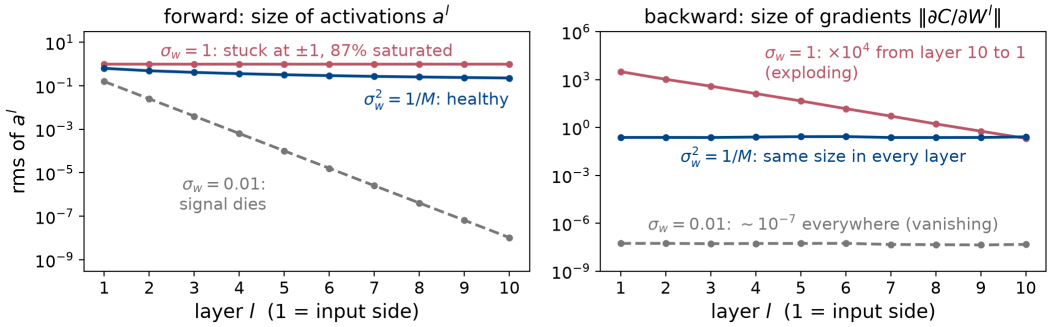
$$\sigma_w^2 = \frac{1}{M}\ (\tanh), \qquad \sigma_w^2 = \frac{2}{M}\ (\mathrm{ReLU})$$

ReLU sets half of its inputs to zero, so $\overline{a^2} = \frac{1}{2}\mathrm{Var}(z)$: hence the factor 2 ("He initialisation"). Sigmoid: $\sigma' \leq \frac{1}{4}$, so with $1/M$ the gradients still shrink layer by layer.

The right scale depends on the *activation function*, and it decides whether *gradients* survive the backward pass.

The wrong scale breaks a deep network before training starts

A deep network: 256 inputs → **10 hidden layers, each with 256 tanh units** → one linear output unit, no biases. Input: 500 random Gaussian vectors; targets: random numbers; cost $C = \frac{1}{2}\overline{(a^L - y)^2}$. Nothing is trained: one forward and one backward pass.



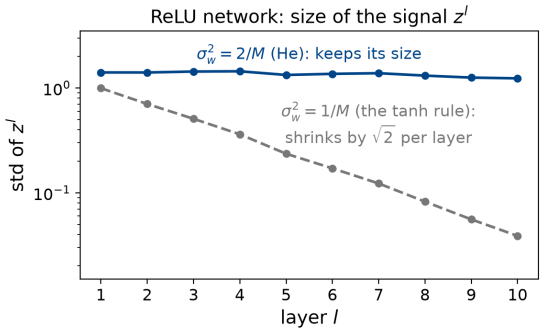
- **Too large:** saturated units and exploding gradients.
- **Too small:** the signal dies and the gradients vanish.
- Either way, gradient descent cannot train the network.
- $\sigma_w^2 = 1/M$**:** signal and gradients keep their size through all 10 layers.

The right scale depends on the activation function

The same network with **ReLU** instead of tanh, initialised with the rule that worked for tanh, $\sigma_w^2 = 1/M$:

- The signal shrinks by $\sqrt{2}$ in *every* layer: $1/23$ of its size after 10 layers, $\sim 10^{-7}$ after 50.
- No error message. The network just trains slowly, or not at all.

Why: ReLU sets half of its inputs to zero, so it passes on only half the variance. He initialisation, $\sigma_w^2 = 2/M$, makes up for it.



Take-away

An initialisation rule belongs to an activation function. Change the activation, and check the scale again.

Output activation and cost function come in pairs

Task	Output activation	Cost (one sample)	$\boldsymbol{\delta}^L$
regression, $y \in \mathbb{R}$	linear, $a = z$	$\frac{1}{2}(a-y)^2$	
binary, $y \in \{0,1\}$	sigmoid, $a = \sigma(z)$	$-y \ln a - (1-y)\ln(1-a)$	
K classes, $\boldsymbol{y}$ one-hot	softmax, $a_k = \frac{e^{z_k}}{\sum_l e^{z_l}}$	$-\sum_k y_k \ln a_k$	

- The **output activation** gives the right *range*: any real number, a probability, or K probabilities that sum to one.
- The **cost** matches it: each is the negative log-likelihood of a Gaussian, Bernoulli or categorical distribution.

For these pairs, $\delta^L = f_L'(z^L)\,\partial C/\partial a^L$. What happens to f_L'?

Output activation and cost function come in pairs

Task	Output activation	Cost (one sample)	$\boldsymbol{\delta}^L$
regression, $y \in \mathbb{R}$	linear, $a = z$	$\frac{1}{2}(a-y)^2$	$a - y$
binary, $y \in \{0, 1\}$	sigmoid, $a = \sigma(z)$	$-y \ln a - (1-y)\ln(1-a)$	$a - y$
K classes, $\boldsymbol{y}$ one-hot	softmax, $a_k = \frac{e^{z_k}}{\sum_l e^{z_l}}$	$-\sum_k y_k \ln a_k$	$\boldsymbol{a} - \boldsymbol{y}$

- The **output activation** gives the right *range*: any real number, a probability, or K probabilities that sum to one.
- The **cost** matches it: each is the negative log-likelihood of a Gaussian, Bernoulli or categorical distribution.

Softmax is not element-wise: there, f' is the Jacobian $\partial a_k / \partial z_j$, but the same cancellation happens.

For these pairs, $\delta^L = f_L'(z^L)\, \partial C / \partial a^L$. What happens to f_L'?

f_L' cancels

For all three pairs, f_L' *cancels* against $\partial C / \partial a$. Sigmoid: $\sigma' = a(1-a)$ and $\frac{\partial C}{\partial a} = \frac{a-y}{a(1-a)}$, so

$$\boxed{\boldsymbol{\delta}^L = \boldsymbol{a}^L - \boldsymbol{y}}$$ prediction minus target.

What goes wrong with a mismatched pair: sigmoid + squared error

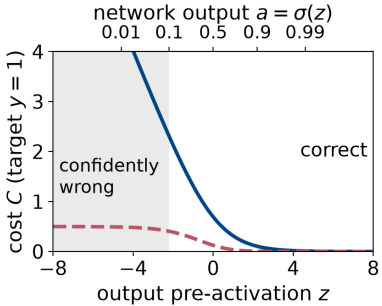
Sigmoid output $a = \sigma(z)$, target $y = 1$:

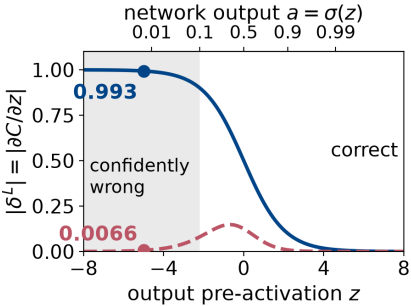
cross entropy: $\delta^L_{\text{CE}} = a - 1$

squared error: $\delta^L_{\text{SE}} = (a-1)\underbrace{a(1-a)}_{\sigma'(z)}$

Here σ' does *not* cancel, and $\sigma' \to 0$ as $a \to 0$.
At $z = -5$: δ^L_{SE} is **150× smaller** than δ^L_{CE}.

Squared error: the more confidently wrong, the *less* it learns.



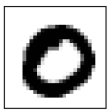


cross entropy: $C = -\ln a$, $\delta^L = a - 1$

squared error: $C = \frac{1}{2}(a-1)^2$, $\delta^L = (a-1)\,a(1-a)$

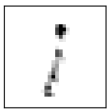
Quick question 2: which image trains the network?

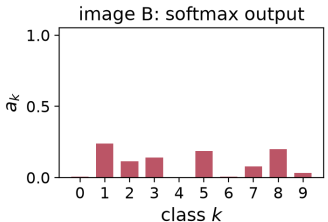
label 0





label 1



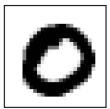


Question

A network trained on handwritten digits (softmax + cross entropy) sees images A and B. The error $\boldsymbol{\delta}^L = \boldsymbol{a}^L - \boldsymbol{y}$ has one component per class $0, \ldots, 9$; $\boldsymbol{y}$ is the one-hot label. Which image changes the weights more?

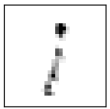
Quick question 2: which image trains the network?

label 0





label 1





Question

A network trained on handwritten digits (softmax + cross entropy) sees images A and B. The error $\boldsymbol{\delta}^L = \boldsymbol{a}^L - \boldsymbol{y}$ has one component per class $0, \ldots, 9$; $\boldsymbol{y}$ is the one-hot label. Which image changes the weights more?

Answer

A, label 0: $\boldsymbol{a} \approx \boldsymbol{y} = (1, 0, \ldots, 0)$, so $\boldsymbol{\delta}^L \approx \mathbf{0}$: nothing to learn.

B, label 1: $\boldsymbol{y} = (0, 1, 0, \ldots, 0)$.

- $\delta_1 = a_1 - 1 = 0.24 - 1 = -0.76$: the only negative component.
- Every other $\delta_k = a_k - 0 = a_k > 0$.

Raise the score of class 1, lower the rest. The network learns from what it gets wrong or is unsure about.

The universal approximation theorem

In words

A network with **one hidden layer** and a **non-linear** (non-polynomial) activation can approximate **any continuous function** on a bounded domain **as accurately as we want**, if the hidden layer is wide enough.

What it says

- A network *can* represent the function we are after.
- The non-linearity is essential.

What it does *not* say

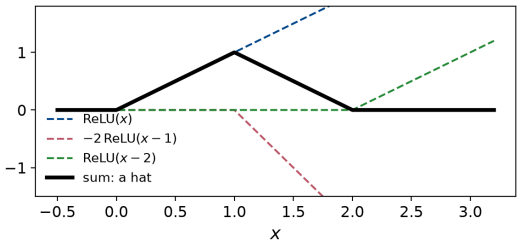
- *How many* units are needed.
- Whether training *finds* the weights.
- How well it generalises to new data.

Why deep networks, then? For many functions, depth does the same job with far fewer units.

Why it works, in one dimension: ReLU units are kinks

One ReLU unit $v\,\mathrm{ReLU}(wx + b)$ is zero, then a straight line: one *kink*, at $x = -b/w$. Three units make a "hat":

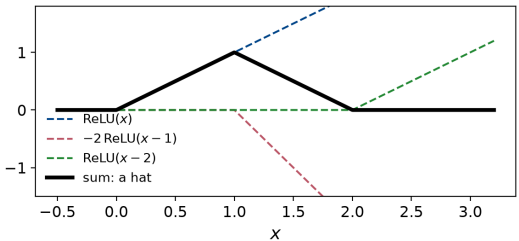
$$\mathrm{ReLU}(x) - 2\,\mathrm{ReLU}(x-1) + \mathrm{ReLU}(x-2)$$



Why it works, in one dimension: ReLU units are kinks

One ReLU unit $v\,\mathrm{ReLU}(wx + b)$ is zero, then a straight line: one *kink*, at $x = -b/w$. Three units make a "hat":

$$\mathrm{ReLU}(x) - 2\,\mathrm{ReLU}(x-1) + \mathrm{ReLU}(x-2)$$

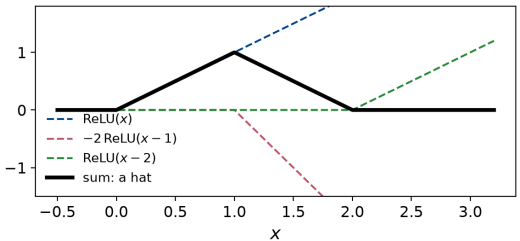


Any function F: one hat of height $F(x_i)$ at each point x_i. Their sum connects the dots $(x_i, F(x_i))$ with straight lines. More points, smaller error.

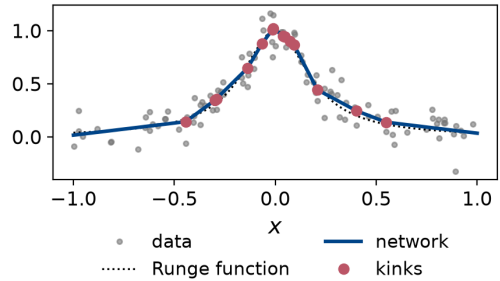
Why it works, in one dimension: ReLU units are kinks

One ReLU unit $v\,\mathrm{ReLU}(wx + b)$ is zero, then a straight line: one *kink*, at $x = -b/w$. Three units make a "hat":

$$\mathrm{ReLU}(x) - 2\,\mathrm{ReLU}(x-1) + \mathrm{ReLU}(x-2)$$



Any function F: one hat of height $F(x_i)$ at each point x_i. Their sum connects the dots $(x_i, F(x_i))$ with straight lines. More points, smaller error.



Training finds this by itself: a network with 50 ReLU units fitted to noisy data from the Runge function. The output is piecewise linear, and the kinks (red) sit where the function bends.

Universal approximation: the precise statements

The theorem (Leshno et al. 1993)

Let f be continuous. One-hidden-layer networks

$$G(\boldsymbol{x}) = \sum_{j=1}^{N} v_j \, f(\boldsymbol{w}_j^T \boldsymbol{x} + b_j)$$

can approximate every continuous F on a compact $K \subset \mathbb{R}^d$ to any accuracy ϵ, $\sup_{\boldsymbol{x} \in K} |F(\boldsymbol{x}) - G(\boldsymbol{x})| < \epsilon$, **if and only if f is not a polynomial**.

- Earlier versions: Cybenko 1989 (sigmoids), Hornik 1991 (any bounded, non-constant f; also in mean square).
- Why not a polynomial: if f has degree p, so has G, for *any* N.

How many units?

- **1D, ReLU:** the hat construction is the piecewise-linear interpolant. For $|F'| \le L$ on $[a, b]$: $\sup_x |F - G| \le \dfrac{L(b-a)}{2N}$.
- **d dimensions, grid-like constructions:** error $\sim N^{-1/d}$, so $\sim \epsilon^{-d}$ units: the curse of dimensionality.
- **Smooth enough** F (Barron 1993): if $C_F = \int |\omega| \, |\hat{F}(\omega)| \, d\omega < \infty$, a sigmoid network reaches mean-square error $\lesssim C_F^2 / N$, *independent of* d.
- **Depth:** some functions need exponentially many units in one hidden layer, but only polynomially many in a deep network (Telgarsky 2016).

Still nothing about whether gradient descent *finds* these weights.

From fitting data to solving equations

Universal approximation: a network $N(x)$ can represent any function. So it can also represent the *solution* of a differential equation. But what do we train it on?

So far: fit data

Targets y_i are given. Cost: how far the output is from them,

$$C = \frac{1}{n} \sum_i \big(N(x_i) - y_i\big)^2$$

Now: solve $g' = -\alpha g$

No targets: the solution is unknown. Cost: how badly the output violates the *equation*,

$$C = \frac{1}{n} \sum_i \big(N'(x_i) + \alpha N(x_i)\big)^2$$

Network, backpropagation and optimiser stay the same; only the cost changes. This is a **physics-informed neural network** (PINN).

Take-aways

1. Why the activation function matters: we need N' and N''
2. Boundary conditions: built in (hard) or as penalties (soft)
3. Against standard methods: Euler, Runge–Kutta, finite differences
4. Equation + data: inverse problems, unknown parameters

Learning the equation: the residual is the cost

Problem: $g'(x) + \alpha g(x) = 0$, $g(0) = g_0$. The network learns the *equation*: the cost measures how badly its output violates it.

Three ingredients:

1. A **trial solution** that satisfies the conditions for *any* network N: $g_t(x) = h(x) + B(x)\,N(x)$, where h satisfies the conditions and $B = 0$ where they are imposed.
2. The **residual**: the left-hand side for g_t, $r(x) = g_t'(x) + \alpha\, g_t(x)$; zero for the exact solution.
3. The **cost**: the mean squared residual at points x_i we choose ourselves (*collocation points*),

$$C = \frac{1}{n}\sum_{i=1}^{n} r(x_i)^2.$$

What is new

The residual contains g_t', so we need $\partial N/\partial x$: a derivative with respect to the *input*, not the weights. It is just as easy to compute as the derivatives with respect to the weights.

What is not new

Network, optimiser and backpropagation for the weights. It is regression with "target" $r = 0$.

Example: exponential decay

$g' + \alpha g = 0$, $g(0) = g_0$, with $\alpha = 2$, $g_0 = 10$.
Exact solution: $g = g_0 e^{-\alpha x}$.

On the board: Exponential decay

Build g_t, compute g_t', write the residual and the cost.

Example: exponential decay

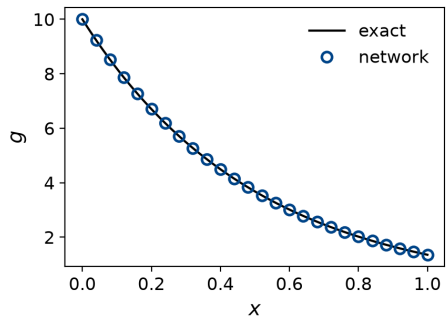
$g' + \alpha g = 0$, $g(0) = g_0$, with $\alpha = 2$, $g_0 = 10$.
Exact solution: $g = g_0 e^{-\alpha x}$.

On the board: Exponential decay

Build g_t, compute g_t', write the residual and the cost.

$$g_t = g_0 + x\,N(x), \qquad r = \underbrace{N + x\,N'}_{g_t'} + \alpha\,(g_0 + x\,N)$$

$g_t(0) = g_0$ whatever the network does.



Two hidden layers of 40 tanh units, 50 collocation points: max error 3×10^{-3}.

The activation function matters even more now

Poisson	diffusion	wave
$g''(x) + f(x) = 0$	$\partial_t g - \partial_x^2 g = 0$	$\partial_t^2 g - c^2\,\partial_x^2 g = 0$

All three contain a *second* derivative, so we need $N''(x)$.

On the board: Second derivatives

Differentiate a one-hidden-layer network $N(x) = \sum_j v_j\, f(w_j x + b_j) + c$ twice with respect to x.

The activation function matters even more now

Poisson	diffusion	wave
$g''(x) + f(x) = 0$	$\partial_t g - \partial_x^2 g = 0$	$\partial_t^2 g - c^2\,\partial_x^2 g = 0$

All three contain a *second* derivative, so we need $N''(x)$.

On the board: Second derivatives

Differentiate a one-hidden-layer network $N(x) = \sum_j v_j\, f(w_j x + b_j) + c$ twice with respect to x.

$$N''(x) = \sum_j v_j\, w_j^2\, f''(w_j x + b_j)$$

ReLU: $f'' = 0$, so $N'' = 0$

A ReLU network is piecewise linear: *no curvature*, for any weights. It cannot solve $g'' + f = 0$.

$\max |N''|$ of a randomly initialised network:

tanh	2.9
GELU	1.2
sigmoid	0.06
ReLU	**0**

We use tanh: smooth, and $f' = 1 - f^2$, $f'' = -2ff'$ are cheap.

Boundary conditions built into the trial solution

Recipe: $g_t = h(x) + B(x)\,N(x)$, where h satisfies the conditions and $B = 0$ wherever a condition is imposed.

Conditions	Trial solution g_t
$g(0) = a,\ g(1) = b$	$a + (b-a)x + x(1-x)\,N$
$g(0) = g_0,\ g'(0) = v_0$	$g_0 + v_0 x + x^2 N$

Check one row: put $x = 0$ and $x = 1$ (for the last row, also differentiate once). The N term drops out every time.

The good

The conditions hold *exactly*, for any weights: at the start, and after every step. The optimiser only has to solve the equation.

The bad

g_t must be designed by hand for every problem. A curved boundary in 2D or 3D: impractical.

Two test problems, three standard methods

Logistic growth:

$$g' - 2g(1-g) = 0, \qquad g(0) = 1.2.$$

Poisson:

$$g'' + (3x + x^2)e^x = 0, \qquad g(0) = g(1) = 0.$$

Standard methods work on a grid with spacing Δt (or Δx).

- **Forward Euler**: follow the slope for one step, $g_{n+1} = g_n + \Delta t\, g'(t_n)$. Error $\propto \Delta t$.
- **RK4** (Runge–Kutta): four slope evaluations per step. Error $\propto \Delta t^4$.
- **Finite differences (FD)**: $g''(x_i) \approx (g_{i+1} - 2g_i + g_{i-1})/\Delta x^2$ turns the equation into a linear system. Error $\propto \Delta x^2$.

Two test problems, three standard methods

Logistic growth:

$$g' - 2g(1-g) = 0, \qquad g(0) = 1.2.$$

Poisson:

$$g'' + (3x + x^2)e^x = 0, \qquad g(0) = g(1) = 0.$$

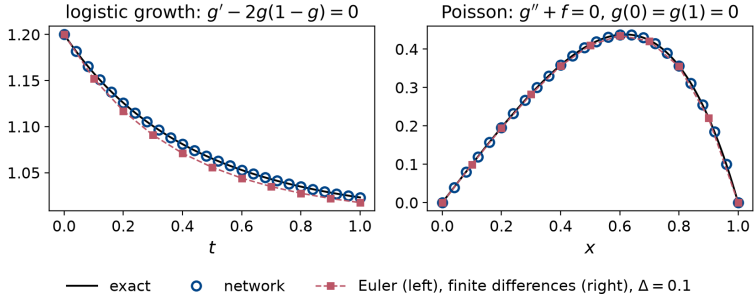
Standard methods work on a grid with spacing Δt (or Δx).

- **Forward Euler**: follow the slope for one step, $g_{n+1} = g_n + \Delta t\, g'(t_n)$. Error $\propto \Delta t$.
- **RK4** (Runge–Kutta): four slope evaluations per step. Error $\propto \Delta t^4$.
- **Finite differences (FD)**: $g''(x_i) \approx (g_{i+1} - 2g_i + g_{i-1})/\Delta x^2$ turns the equation into a linear system. Error $\propto \Delta x^2$.

Guess first

Network or standard method: which is more *accurate*? Which is *faster*?

Results against standard methods



	method	max error	time
logistic	Euler, $\Delta t = 0.01$	9×10^{-4}	0.04 ms
	RK4, $\Delta t = 0.01$	2×10^{-10}	0.14 ms
	network	1×10^{-4}	0.5 s
Poisson	FD, $\Delta x = 0.01$	2×10^{-5}	0.12 ms
	network	1×10^{-5}	0.6 s

Network: 4000 training steps of 0.13–0.15 ms each. Plots use spacing 0.1, so the grid points are visible.

Conclusion

The network beats Euler, but RK4 is far more accurate. And the network is thousands of times slower. For simple equations like these the standard methods win.

A partial differential equation: diffusion

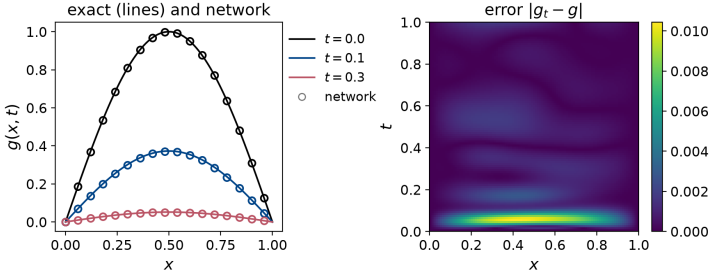
$$\frac{\partial g}{\partial t} - \frac{\partial^2 g}{\partial x^2} = 0, \qquad g(0,t) = g(1,t) = 0, \quad g(x,0) = \sin \pi x.$$

Three conditions, one trial solution:

$$\boxed{g_t = (1-t)\sin \pi x + x(1-x)\, t\, N(x,t)}$$

The network now has *two inputs*, $N(x,t)$.
Nothing else changes.

Check: $t = 0$ gives $\sin \pi x$; $x = 0$ and $x = 1$ give 0.



Error exactly zero on the three edges (built into g_t), largest near $t = 0$ where the solution changes fastest: max 1×10^{-2}, after 1.2 s of training (800 steps). Explicit finite differences with $\Delta x = 0.05$: max error 1.5×10^{-3} in 2 ms.

Soft constraints: conditions as penalties

Instead of building g_t by hand: let the network itself be the solution, $u(x,t) \approx N(x,t)$, and add the conditions to the cost as penalties:

$$L = L_{\mathrm{PDE}} + \lambda\,(L_{\mathrm{IC}} + L_{\mathrm{BC}})$$

$$L_{\mathrm{PDE}} = \tfrac{1}{n}\textstyle\sum_i (\partial_t N - \partial_x^2 N)^2 \qquad \text{inside the domain}$$
$$L_{\mathrm{IC}} = \tfrac{1}{n}\textstyle\sum_i (N(x_i,0) - \sin \pi x_i)^2 \qquad \text{at } t = 0$$
$$L_{\mathrm{BC}} = \tfrac{1}{n}\textstyle\sum_i (N(0,t_i)^2 + N(1,t_i)^2) \qquad \text{at } x = 0, 1$$

A new condition is just a new term. λ weighs the terms, like the penalty in Ridge regression.

	RMSE	error at $t = 0$
hard (g_t)	1.8×10^{-3}	0 exactly
soft, $\lambda = 10$	6.5×10^{-3}	6×10^{-3}

Hard or soft?

- If you *can* build a trial solution, do: exact and more accurate.
- Soft is flexible, but the conditions hold only approximately.
- λ must be tuned. Too large, and the optimiser neglects the equation.

Quick question 3: a small residual, a wrong answer

Question

We solve $\partial_t u - \partial_x^2 u = 0$ with the soft cost $L = L_{\mathrm{PDE}} + \lambda(L_{\mathrm{IC}} + L_{\mathrm{BC}})$, but use a **ReLU** network $u = N(x, t)$. What does training converge to?

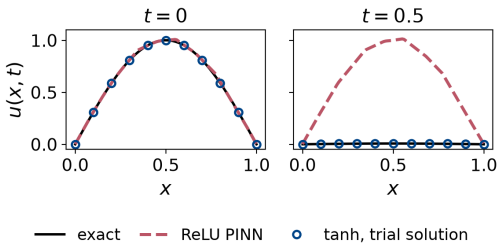
Quick question 3: a small residual, a wrong answer

Question

We solve $\partial_t u - \partial_x^2 u = 0$ with the soft cost $L = L_{\mathrm{PDE}} + \lambda(L_{\mathrm{IC}} + L_{\mathrm{BC}})$, but use a **ReLU** network $u = N(x, t)$. What does training converge to?

Answer

- ReLU gives $\partial_x^2 N = 0$ at every sample, so L_{PDE} only asks for $\partial_t N = 0$.
- The network learns $u \approx \sin \pi x$ for all t: it never diffuses. Small cost (6×10^{-4}), wrong answer: RMSE 0.65 (vs. 1.8×10^{-3}).
- **A small residual is not a solution.** Use smooth activations, and check against something you know.



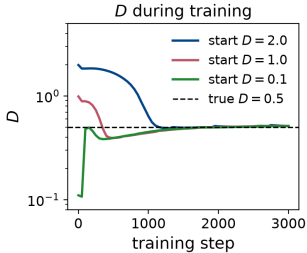
Inverse problems: find the unknown parameters of the equation from data

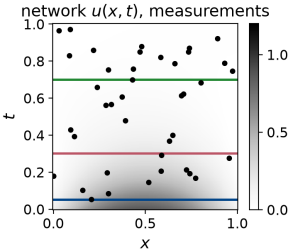
$\partial_t u - D\,\partial_x^2 u = 0$ with D **unknown**. No initial or boundary conditions, only 40 noisy measurements y_i. Treat D as one more trainable parameter, and fit the equation and the data together:

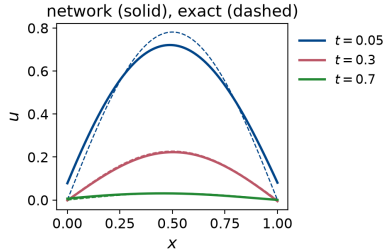
$$L(N, D) = \tfrac{1}{n}\sum_j \left(\partial_t N - D\,\partial_x^2 N\right)_j^2 + \lambda \tfrac{1}{40}\sum_i \left(N(x_i, t_i) - y_i\right)^2$$

A standard solver needs D to run; here the network finds D and the solution u in one training run.

True $D = 0.5$. Three different starting guesses all give $D \approx 0.51$. **But** with five times more noise: $D = 0.69$. A recovered parameter needs an error bar.







Neural networks versus standard methods

Advantages

- **No mesh**: complicated geometries are easy.
- **High dimensions**: a mesh grows exponentially with the dimension, random points do not.
- **The solution is a function**: evaluate and differentiate it anywhere.
- **Non-linear equations** need no extra work.
- **Data and equations together**: unknown parameters, inverse problems.

Disadvantages

- **Slow**: thousands of training steps vs. milliseconds.
- **No error guarantees** like "error $\propto \Delta x^2$".
- **Local minima**: can converge to a wrong answer.
- **Hyperparameters**: architecture, learning rate, λ.
- **Smooth activations needed**: no ReLU for second-order equations.

Bottom line: simple problems in one or two dimensions: use standard methods. Networks pay off in high dimensions, complicated geometries and inverse problems. **Always compare with something.**

Read more

Parts 1–3: training a network

- **Course book** (M. Hjorth-Jensen, *Machine Learning and the Physical Sciences*, the lecture notes): Section 8.8 (backpropagation; 8.8.1 scalar chain, 8.8.4 four equations), 8.6 (activations), 8.9 (vanishing gradients), 8.10 (initialisation), 8.7 and 8.11 (cost and output), 8.4 (universal approximation).
- **Goodfellow et al.**, *Deep Learning*: 6.5 (backpropagation), 6.3 (hidden units), 6.2.1–6.2.2 (cost and output units), 8.2.5 (vanishing gradients), 8.4 (initialisation), 6.4.1 (universal approximation).
- **Mehta et al.**, *A high-bias, low-variance introduction to ML for physicists*, Phys. Rep. **810** (2019): Section IX.A–C.
- **Hastie et al.**, *Elements of Statistical Learning*: 11.3–11.4, 11.5.1 (starting values).

Part 4: differential equations

- **Course book:** Chapter 9: Sections 9.1–9.3 (trial solution, $\partial N/\partial x$ and why not ReLU, decay), 9.5–9.7 (Poisson, PDEs, diffusion), 9.9 (when it is worth doing), 9.10 (PINNs, inverse problems).
- I. E. Lagaris, A. Likas, D. I. Fotiadis, IEEE Trans. Neural Netw. **9**, 987 (1998): trial solutions.
- M. Raissi, P. Perdikaris, G. E. Karniadakis, J. Comput. Phys. **378**, 686 (2019): PINNs.

Code: every figure and number on these slides comes from `week42monday.ipynb`.

Summary

- **Backpropagation** is the chain rule: compute the errors $\boldsymbol{\delta}^l$ from the output backwards; weight gradient = error $\times$ incoming signal.
- **Hidden activations** must be non-linear; training sees f'.
- **Output activation and cost come in pairs**, and then $\boldsymbol{\delta}^L = \boldsymbol{a}^L - \boldsymbol{y}$.
- **Universal approximation**: a wide enough network can represent any continuous function, but the theorem says nothing about size or training.
- **Differential equations (PINNs)**: the residual is the cost. Conditions go into the trial solution (hard) or the cost (soft). Use smooth activations. Flexible, but slower than standard methods and without their guarantees.

Next

Tuesday: backpropagation in code. **Exercises week 42:** Friday October 16. **Project 2:** November 6.